案例 02 · 把 AI 从「对话框」变成「工作台」

角色：独立完成（环境部署 → 排障 → 能力配置）

时间：2026.09

技术：Node.js / 命令行 / 插件化 AI Harness（DeepSeek Harness）/ 依赖与构建机制

可验证材料：`交接.md`（记录了完整部署链路与排障过程）

一、问题

我一开始用网页版 AI 帮我做翻译、整理资料、写东西，但很快发现三个毛病：

- **每次都要重新交代背景** —— 同一个行业、同一套术语，说一遍又一遍
- **同一件事两次结果不一样** —— 没有固定的做事方式
- **改一点就要从头来** —— 想让它换个风格，得把上文重贴一遍

本质问题：**我在把 AI 当搜索框用，而不是当工具用。**

二、我做了什么

1. 选了可本地部署的方案

不满足于网页版对话框，改成在本地装一套 AI Harness（DeepSeek Harness），把「能力」做成可装配的插件，而不是每次临场描述。

2. 途中真实的四个坑（这才是我学到东西的地方）

坑	现象	我的处理
网络受限	`git clone` 连不上 github.com，官方推荐的 `github:` 安装方式直接失败	换用 `api.github.com` 的 tarball 接口拉源码，绕开被墙的域名
供应链陷阱	npm 上确实存在一个**同名不同作者**的包，仓库指向另一个项目（19 star），而我要的是那个 7000+ star 的	对比 `package.json` 的 `repository` 字段确认**不是同一个东西**，坚决不用裸包名安装
构建被拦	pnpm 默认拦截自带构建脚本的依赖，导致插件源码没有被编译出 `lib/`	在 `pnpm-workspace.yaml` 的 `allowBuilds` 里显式放行对应包名
装配链路不生效	依赖装上了，但插件没被加载	读 DSH 的启动代码，搞清 bundle 必须进 `dsh.profile.bundles` 才会被装配

3. 最终固化成一套环境

- 任务路由：按任务类型切换不同的思考策略
- 工具插件按需装载、热重载（改配置不用重启）
- 出问题能自愈（重复条目、失效链接）

三、结果

- 同类任务的重复交代成本降为 **0**
- 用它完成的产出：【待填：X 篇技术文档翻译 / Y 份资料整理 / Z 小时节省】
- 更重要的是：**我搞清了「一个插件从装到生效」的完整链路**，不再只是「点开网页就能用」的使用者

四、这个案例能证明我什么

- **英文技术文档阅读能力**：这些工具文档全是英文，我读懂了才装得上
- **排障能力**：网络、依赖、构建、装配，四类问题的排查思路
- **风险意识**：识别出同名包的供应链陷阱，没盲目 `npm install`
- **不依赖图形界面**：能碰命令行

> 对英语专业来说是加分项：**证明我不是只能做文字工作，我能把工具用起来。**

五、面试官可能追问

- **「这些命令你是照着教程敲的吗？」**

→ 诚实答：教程只给了两条命令，装不上。后面是我自己看报错、查文档、翻源码解决的。**能说清每个坑的判断依据，比说「我会」有用。**

- **「一个英语专业的为什么要折腾这个？」**

→ 答：因为我想投的是 AI 落地岗，如果我自己都只停在「用对话框」，我凭什么跟客户说 AI 能解决他的问题。

- **「你觉得这套东西最大的局限是什么？」**

→ 答：它解决的是「稳定性」和「不重复劳动」，解决不了「我不知道要做什么」。**需求判断还是人的事**——这也是我想做 FDE 的原因。